CocoConnect

Technical Feasibility Study

Final Version

Client: Coconut Growers Association of Sri Lanka · Group 38

Prepared by: S.N. Vidanage (2023/IS/108)

Contents

Contents 2

Document History 3

1. Executive Summary 3

2. Final Technology Decisions 3

3. Evaluation of Development Tools 3

4. Module Feasibility 3

5. Resource Requirements 4

6. Technical Limitations and Solutions 4

7. Final Risk Assessment 4

8. Conclusion and Recommendation 5

Document History

Version	Content
1	Overall feasibility of the stack, modules, integrations, cost, security and risks
2	Implementation feasibility of each marketplace function, component reuse and build order
Final	This document: the consolidated findings, final technology decisions, evaluation of tools, resource requirements, technical limitations with their solutions, and the final recommendation

1. Executive Summary

CocoConnect is technically feasible and ready for implementation. The final stack is Flutter for the mobile app, Node.js with Express.js for the backend, and PostgreSQL hosted on Supabase, with JWT and bcrypt for security and OpenStreetMap for maps. All core modules are rated high feasibility. Negotiation, offline harvest recording and the service calendar are rated medium and have defined solutions. Online payment is the only feature not feasible within the project's time and budget, and is deferred.

2. Final Technology Decisions

Layer	Option	Decision	Reason
Mobile app	Flutter	**Adopted**	One codebase for Android and iOS; consistent UI; built-in localisation
Mobile app	React Native / native	**Rejected**	UI differs per platform / two codebases
Backend	Node.js + Express.js	**Adopted**	Lightweight REST API; large package ecosystem; familiar language
Backend	Django / Spring Boot	**Rejected**	Heavier setup than the project needs
Database	PostgreSQL on Supabase	**Adopted**	Relational data, transactions, free hosted tier shared by the team
Database	MongoDB / Firebase	**Rejected**	Weak support for the strongly related data
Security	JWT + bcrypt	**Adopted**	Token login suited to mobile; hashed passwords
Maps	OpenStreetMap + Nominatim	**Adopted**	Free, no billing account
Maps	Google Maps API	**Rejected**	Needs a billing account
Payments	Online payment gateway	**Rejected**	Cost, merchant registration and time; deferred

3. Evaluation of Development Tools

Tool	Purpose	Assessment
Flutter SDK + Dart	Build the app	**Feasible**
Android Studio + emulator	Run and test the app without a phone	**Feasible**
Visual Studio Code	Edit app and backend code	**Feasible**
Node.js + npm	Run the backend and install packages	**Feasible**
Supabase dashboard	View tables and run SQL changes	**Feasible**
Swagger UI	Document and try every API route	**Feasible**
Git + GitHub	Version control with a branch per member	**Feasible**
Figma / Google Stitch	Wireframes and UI design	**Feasible**
Firebase Cloud Messaging	Push notifications	**Feasible with conditions**

4. Module Feasibility

Module	Feasibility	Basis
User management and roles	**High**	Standard login with hashed passwords, tokens and role checks

Module	Feasibility	Basis
Location at sign-up and distance filters	**High**	Coordinates stored per user; distance calculated on the server
Product marketplace and request matching	**High**	Standard database queries with filters
Negotiation chat and order creation	**Medium**	Offer states and database transactions keep both sides consistent
Orders hub and deal completion	**High**	Reads and updates of the orders table; admin review for price changes
Alerts and market price	**High**	Built from existing data; refreshed regularly
Service marketplace and machinery map	**High**	Reuses the marketplace patterns and the map component
Service calendar	**Medium**	Booking dates with checks for overlaps
Ratings and reviews	**High**	Simple records linked to completed orders and jobs
Offline harvest recording	**Medium**	Local storage with a sync status, uploaded when online
Online payment	**Low**	Deferred

5. Resource Requirements

Resource	Requirement	Availability
People	Six members: development, business analysis and testing roles	Available
Time	Two semesters, delivered in sprints	Available
Developer computers	Able to run Android Studio and the emulator (8 GB RAM or more recommended)	Available
Test devices	Android emulator; Android phones for field testing	Available
Cloud services	Supabase free tier for the database	Available
Budget	Free and open-source tools; no paid services in this release	Within budget
Knowledge	Flutter, Node.js and SQL, supported by documentation and small prototypes	Available with learning

6. Technical Limitations and Solutions

Limitation / challenge	Solution
Weak internet and GPS in rural estates	Offline harvest recording with later sync; pick location on a map when GPS is weak; clear timeout messages
No push notifications in the first release	Alerts tab with badge counts; screens refresh regularly and when reopened
Keeping offers consistent when both sides send prices	Offer statuses (pending, accepted, rejected, replaced); only the receiver may accept; database transaction on accept
Several members changing the same shared database	Numbered SQL scripts that only add to existing tables and are safe to run again
Testing without damaging shared data	Automated tests on an in-memory database
Separately built modules must fit together	API contracts in Swagger; integration branch; frequent merges
Windows build problems (Kotlin, symbolic links)	kotlin.incremental=false; only the Android geolocator package
Emulator cannot reach localhost	Use 10.0.2.2, or set the address at run time
Protecting secrets	Database address and token secret kept in a private .env file, never in Git
Users with low digital literacy	Simple screens, three languages, large buttons, clear messages

7. Final Risk Assessment

Risk	Likelihood	Impact	Status after mitigation
Rural connectivity	High	High	Medium
Low digital literacy	High	Medium	Low
Inconsistent offers between users	Medium	High	Low

Risk	Likelihood	Impact	Status after mitigation
Integration between members' modules	Medium	High	Medium
Free-tier service limits	Low	Medium	Low
Scope growth	Medium	High	Medium

8. Conclusion and Recommendation

Recommendation: proceed with implementation. The technology stack, tools and resources are in place, every core module is feasible, and each known limitation has a planned solution.

- Build in this order: users and location → marketplace and matching → negotiation and orders → alerts → service marketplace → ratings → offline harvest.
- Share the Flutter project configuration and dependency files with all members before coding starts.
- Document each API in Swagger before connecting screens.
- Keep online payment and push notifications for a later phase.